

1: DaT Parkinson's Challenge Description

DaT Parkinson's Challenge — Problem Description

Source: <https://www.drivendata.org/competitions/311/dat-parkinsons-challenge/page/990/>

Your goal is to develop machine learning models that predict the probability that a dopamine transporter (DaT) scan examination is **abnormal** rather than **normal**.

In this challenge, you will build models that distinguish normal from abnormal DaT scans using a large multicenter dataset collected and annotated by French experts. Successful solutions should generalize across institutions, scanners, and patient populations, and could help improve the consistency, accessibility, and scalability of DaT scan interpretation.

Dataset

The imaging data are provided as three-dimensional DaT scan reconstructions in compressed Neuroimaging Informatics Technology Initiative (NIfTI) format (`.nii.gz`). Each examination is a single 3D volume, and each patient is represented by one file. The filename (minus the `.nii.gz` extension) is the `uid` that links each image to its label.

In addition, participants are welcome to incorporate external datasets for training; however, use of external data is subject to important exceptions and caveats (see External Data section on the Home page, and the full Rules).

Participants are not allowed to share the competition data or use it for any purpose other than this competition. Participants cannot send the data to any third-party service or API, including, but not limited to, OpenAI's ChatGPT, Google's Gemini, or similar tools. For complete details, review the competition rules.

Answer

The target

OOF log loss < 0.25,

OOF AUROC > 0.93

may be achievable, but it cannot be guaranteed without knowing the dataset distribution and the exact validation protocol. The most likely causes of failing to reach it are data leakage, inconsistent spatial preprocessing, overconfident probabilities, and training a CNN from scratch on too few independent examinations.

Solution

1. Define the validation correctly

Use an **out-of-fold (OOF) evaluation** that matches the competition setting.

- Read the label from the training metadata and join it to the image using the uid.
- Ensure that each patient appears only once.
- If institution, scanner, acquisition protocol, or site identifiers are available, use a grouped split:

```

from sklearn.model_selection import StratifiedGroupKFold

cv = StratifiedGroupKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

for fold, (train_idx, valid_idx) in enumerate(
    cv.split(X, y, groups=site_or_patient_group)
):
    ...

```

If no valid grouping variable exists, use `StratifiedKFold`. Do not use random slices from the same 3D volume in different folds.

Important: all intensity normalization, feature selection, PCA, resampling, and calibration must be fitted separately inside each training fold. Otherwise, the OOF scores will be optimistic.

2. Standardize the 3D scans

For every `.nii.gz` file:

- Load the NIfTI volume.
- Reorient it consistently, for example to RAS.
- Remove empty background.
- Resample every scan to the same voxel spacing and shape.
- Clip extreme intensities using training-set percentiles.
- Normalize intensity within the brain or striatal region.

A robust normalization is

$$I_{\text{norm}} = \frac{\text{clip}(I, q_1, q_{99}) - \text{median}(I_{\text{brain}})}{\text{IQR}(I_{\text{brain}}) + \epsilon}.$$

Do not calculate the percentiles using the validation fold.

A bounding box around the basal ganglia is preferable to using the entire head. If an anatomical atlas or registration procedure is available, register all scans to a common template before extracting features.

3. Build biologically meaningful features

A strong first model is usually an ensemble of interpretable regional features and image-based features rather than a small CNN trained from scratch.

For each scan, calculate:

- Left and right striatal mean uptake.
- Left and right caudate uptake.
- Left and right putamen uptake.
- Regional standard deviations and quantiles.
- Left-to-right asymmetry.

- Putamen-to-caudate ratios.
- Striatum-to-background ratios.
- Posterior-to-anterior putamen ratios.
- Total striatal uptake.
- Voxel-count or volume measures above several intensity thresholds.
- Histogram, texture, and shape features from the striatal masks.

For a left-right pair, use features such as

$$A_{LR} = \frac{|L - R|}{(L + R)/2 + \epsilon},$$

and

$$R_{PC} = \frac{\text{putamen uptake}}{\text{caudate uptake} + \epsilon}.$$

Use both raw and normalized versions. Parkinsonian abnormalities are often expressed through reduced posterior putamen uptake and increased left-right asymmetry.

If reliable segmentation is unavailable, create several fixed-size central crops and compute the same statistics from each crop.

4. Train strong tabular base models

Use several models with different inductive biases:

- CatBoost or LightGBM on regional and radiomic features.
- Logistic regression with elastic-net regularization.
- XGBoost on selected regional features.
- A small multilayer perceptron only after establishing a strong tabular baseline.

Example CatBoost configuration:

```
from catboost import CatBoostClassifier

model = CatBoostClassifier(
    loss_function="Logloss",
    eval_metric="Logloss",
    iterations=2000,
    depth=5,
    learning_rate=0.02,
    l2_leaf_reg=8,
    random_strength=0.5,
    random_seed=42,
    verbose=False
)
```

Use early stopping on the validation fold. Do not optimize AUROC alone: the competition's log loss requires well-calibrated probabilities.

For logistic regression:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
    ("classifier", LogisticRegression(
        penalty="elasticnet",
        solver="saga",
        l1_ratio=0.15,
        C=0.2,
        max_iter=5000,
        class_weight=None
    ))
])

```

5. Add an image-based model carefully

A CNN trained from scratch on a small medical dataset is prone to overfitting. Prefer one of these approaches:

- A pretrained 2D model applied to axial, coronal, and sagittal projections.
- A 2.5D model using adjacent slices as channels.
- A pretrained 3D encoder with a small classification head.
- Fixed image embeddings followed by logistic regression or CatBoost.

For each volume, generate standardized projections:

$$P_{\text{axial}}(x, y) = \max_z I(x, y, z),$$

with analogous coronal and sagittal projections. Use mean projections as an additional representation, not as the only representation.

Augmentations should be medically plausible:

- Small translations.
- Small rotations.
- Mild intensity scaling.
- Gaussian noise.
- Random cropping around the striatum.

Avoid left-right flipping unless the label is guaranteed to be invariant and the asymmetry features are handled consistently.

6. Use fold-wise model ensembling

Generate OOF predictions from the tabular and image models. Then combine them using a small blending model or a fixed weighted average.

For example,

$$p_{\text{blend}} = wp_{\text{tabular}} + (1 - w)p_{\text{image}}, \quad 0 \leq w \leq 1.$$

Select w using only the OOF predictions. A practical search is over values from 0 to 1 in increments of 0.02. Optimize validation log loss first, while checking AUROC as a secondary metric.

Do not train the blender on predictions from samples that were used to train the corresponding base model. Use strictly OOF predictions.

7. Calibrate the probabilities

A model can have excellent AUROC but poor log loss because its probabilities are too extreme. Fit calibration on OOF predictions:

```
from scipy.special import expit
from scipy.optimize import minimize_scalar
import numpy as np

def logloss_temperature(T, logits, y):
    p = expit(logits / T)
    p = np.clip(p, 1e-6, 1 - 1e-6)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

temperature = minimize_scalar(
    logloss_temperature,
    bounds=(0.25, 5.0),
    method="bounded",
    args=(oof_logits, y)
).x

calibrated_oof = expit(oof_logits / temperature)
```

Temperature scaling preserves the ranking and therefore usually preserves AUROC while improving log loss.

Also test a small probability shrinkage toward the training prevalence π :

$$p_{\text{shrunk}} = (1 - \lambda)p + \lambda\pi.$$

Optimize λ on OOF predictions only. This can substantially improve log loss when the classifier is overconfident.

8. Calculate OOF metrics

```
from sklearn.metrics import roc_auc_score, log_loss

oof_p = np.clip(oof_p, 1e-6, 1 - 1e-6)

auroc = roc_auc_score(y, oof_p)
ll = log_loss(y, oof_p)

print(f"OOF AUROC: {auroc:.4f}")
print(f"OOF log loss: {ll:.4f}")
```

Use the same folds and preprocessing for every experiment. Record:

- Fold-specific AUROC.
- Fold-specific log loss.
- Mean and standard deviation.
- Calibration curve.
- Predictions for difficult and borderline cases.
- Performance by site or scanner, if metadata permit.

A single favorable random split is not sufficient evidence that the model reaches the target.

9. Recommended development sequence

1. Regional uptake features plus regularized logistic regression.
2. Regional features plus CatBoost.
3. Add radiomic and texture features.
4. Add image embeddings or a 2.5D model.
5. Blend tabular and image predictions.
6. Apply temperature scaling or probability shrinkage.
7. Repeat the entire pipeline with several seeds.
8. Train the final models on all labeled data and average test predictions across seeds and folds.

The target

$$\text{OOF log loss} < 0.25, \quad \text{OOF AUROC} > 0.93$$

may be achievable, but it cannot be guaranteed without knowing the dataset distribution and the exact validation protocol. The most likely causes of failing to reach it are data leakage, inconsistent spatial preprocessing, overconfident probabilities, and training a CNN from scratch on too few independent examinations.

2: File Structure Example

```
Files
...
data/
├── niftis/
│   ├── xaji0y6d.nii.gz
│   ├── pbhsahxt.nii.gz
│   └── ...
└── train\\_labels.csv
...
```

Answer

No answer

Solution

1. Correct the file structure

Use one row per NIfTI file in train_labels.csv:

```
project/
├── data/
│   ├── nifti/
│   │   ├── xaji0y6d.nii.gz
│   │   ├── pbhsahxt.nii.gz
│   │   └── ...
│   └── train_labels.csv
├── src/
│   ├── dataset.py
│   ├── model.py
│   ├── train.py
│   └── evaluate.py
├── splits/
│   ├── train.csv
│   ├── val.csv
│   └── oof_predictions.csv
└── configs/
    └── baseline.yaml
```

Example train_labels.csv:

```
file_name,label,patient_id
xaji0y6d.nii.gz,0,P001
pbhsahxt.nii.gz,1,P002
```

The patient_id column is essential. All scans from the same patient must remain in the same fold.

2. Use patient-level stratified cross-validation

A target such as OOF AUROC greater than 0.93 cannot be guaranteed without knowing the data quality and task difficulty. The most important requirement is preventing leakage.

Use **StratifiedGroupKFold**:

```
from sklearn.model_selection import StratifiedGroupKFold

splitter = StratifiedGroupKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

for fold, (train_idx, val_idx) in enumerate(
    splitter.split(
        X=df["file_name"],
        y=df["label"],
        groups=df["patient_id"]
    )
):
    train_df = df.iloc[train_idx].reset_index(drop=True)
    val_df = df.iloc[val_idx].reset_index(drop=True)

    train_df.to_csv(f"splits/fold{fold}_train.csv", index=False)
    val_df.to_csv(f"splits/fold{fold}_val.csv", index=False)
```

Do not perform any of the following before splitting:

- intensity normalization using the complete dataset;
- oversampling;
- feature extraction;
- augmentation;
- preprocessing fitted on all images;
- selection of validation cases.

3. Preprocess the NIfTI volumes consistently

A robust baseline pipeline is:

1. Load the image with nibabel.
2. Reorient to a common orientation.
3. Resample every volume to a common voxel spacing.
4. Crop or pad to a fixed size.
5. Clip extreme intensities using training-fold statistics.

6. Normalize each volume.

Example normalization:

```
import numpy as np

def normalize_volume(volume, low=-1000, high=1000):
    volume = np.nan_to_num(volume)
    volume = np.clip(volume, low, high)
    volume = (volume - low) / (high - low)
    return volume.astype(np.float32)
```

For MRI, fixed Hounsfield-unit limits are inappropriate. Use robust percentile normalization estimated only from the training fold:

```
def robust_normalize(volume):
    volume = np.nan_to_num(volume).astype(np.float32)

    p_low, p_high = np.percentile(volume, [1, 99])
    volume = np.clip(volume, p_low, p_high)

    mean = volume.mean()
    std = volume.std() + 1e-6
    return ((volume - mean) / std).astype(np.float32)
```

If the anatomy occupies only a small region, use a body or foreground mask before resizing. Background-dominated volumes often produce poor likelihood and AUROC.

4. Start with a pretrained 3D model

A strong baseline is a pretrained 3D convolutional network from **MONAI**:

```
import torch
import torch.nn as nn
from monai.networks.nets import DenseNet121

class Classifier3D(nn.Module):
    def __init__(self):
        super().__init__()

        self.backbone = DenseNet121(
            spatial_dims=3,
            in_channels=1,
            out_channels=1
        )

    def forward(self, x):
        return self.backbone(x).flatten(1)
```

If the volumes are too large for a full 3D model, use one of these alternatives:

- 2.5D input consisting of adjacent slices;
- slice-level feature extraction followed by attention pooling;
- a pretrained 2D model applied to representative slices;
- patch-based 3D training with volume-level aggregation.

Use a pretrained model whenever possible. Training a 3D model from scratch usually requires substantially more labeled data.

5. Optimize a loss related to log loss

For binary classification, use logits with **binary cross-entropy**:

```
criterion = nn.BCEWithLogitsLoss()
```

For imbalanced labels, use a positive-class weight computed from the training fold only:

```
n_negative = (train_df["label"] == 0).sum()
n_positive = (train_df["label"] == 1).sum()
```

```
pos_weight = torch.tensor(
    [n_negative / max(n_positive, 1)],
    dtype=torch.float32,
    device=device
)
```

```
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
```

However, `pos_weight` may improve classification balance while making probabilities less calibrated. Since OOF log loss depends directly on probability quality, evaluate both:

```
from sklearn.metrics import log_loss, roc_auc_score
```

```
probabilities = torch.sigmoid(logits).detach().cpu().numpy()
labels = targets.cpu().numpy()
```

```
fold_logloss = log_loss(labels, probabilities, labels=[0, 1])
fold_auc = roc_auc_score(labels, probabilities)
```

For a log-loss-focused competition, ordinary BCE without aggressive class weighting is often a better final model if the classes are not severely imbalanced.

6. Use conservative augmentation

Useful 3D augmentations include:

```
from monai.transforms import Compose, RandFlip, RandGaussianNoise
from monai.transforms import RandAffine, RandAdjustContrast
```

```
train_transforms = Compose([
    RandFlip(prob=0.5, spatial_axis=0),
    RandFlip(prob=0.5, spatial_axis=1),
    RandAffine(
        prob=0.35,
        rotate_range=0.15,
        scale_range=0.10,
        translate_range=0.05,
        padding_mode="border"
    ),
    RandGaussianNoise(prob=0.15, std=0.01),
    RandAdjustContrast(prob=0.15, gamma=(0.8, 1.2))
])
```

Do not use left-right flips when laterality is clinically meaningful. Excessively strong augmentation can damage calibration and increase log loss.

7. Train with early stopping and a scheduler

A suitable baseline configuration is:

```
epochs: 50
batch_size: 2
learning_rate: 0.0001
```

```
weight_decay: 0.00001
optimizer: AdamW
scheduler: cosine
early_stopping_patience: 8
```

Example optimizer:

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-4,
    weight_decay=1e-5
)

scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=50
)
```

Select the best checkpoint using validation log loss, not training loss:

```
if val_logloss < best_logloss:
    best_logloss = val_logloss
    torch.save(model.state_dict(), checkpoint_path)
```

If AUROC improves while log loss worsens, the model is probably becoming overconfident.

8. Generate genuine OOF predictions

For each fold:

```
oof = df.copy()
oof["prediction"] = 0.0

for fold in range(5):
    train_df = pd.read_csv(f"splits/fold{fold}_train.csv")
    val_df = pd.read_csv(f"splits/fold{fold}_val.csv")

    model = Classifier3D().to(device)
    model.load_state_dict(
        torch.load(f"checkpoints/fold{fold}.pth", map_location=device)
    )

    model.eval()

    with torch.no_grad():
        for file_name, image, index in validation_loader:
            logits = model(image.to(device))
            probs = torch.sigmoid(logits).squeeze(1)
            oof.loc[index, "prediction"] = probs.cpu().numpy()
```

Then compute the global, out-of-fold metrics:

```
from sklearn.metrics import log_loss, roc_auc_score

y_true = oof["label"].to_numpy()
y_pred = oof["prediction"].clip(1e-6, 1 - 1e-6)

print("OOF log loss:", log_loss(y_true, y_pred))
print("OOF AUROC:", roc_auc_score(y_true, y_pred))
```

Do not average fold metrics if the folds have substantially different sizes. Compute the metrics once from the concatenated OOF predictions.

9. Calibrate probabilities

Probability calibration is especially important for achieving log loss below 0.25. Fit calibration parameters using validation predictions, not training predictions.

A simple temperature-scaled model is:

$$p_i = \text{sigmoid} \left(\frac{z_i}{T} \right),$$

where z_i is the model logit and $T > 0$ is fitted to minimize validation log loss.

For a final ensemble, average probabilities rather than logits:

$$\hat{p}_i = \frac{1}{K} \sum_{k=1}^K \hat{p}_{i,k}.$$

Then clip only for numerical stability:

$$\hat{p}_i = \min(1 - 10^{-6}, \max(10^{-6}, \hat{p}_i)).$$

Averaging five fold models and, if possible, several random seeds generally improves both calibration and AUROC.

10. Recommended path to the target

Use the following order:

- Verify that every patient appears in exactly one fold.
- Confirm that no duplicated or near-duplicated scans cross folds.
- Establish a 3D pretrained baseline.
- Optimize validation log loss with BCE.
- Add conservative spatial and intensity augmentation.
- Train five patient-level folds.
- Average fold probabilities.
- Apply temperature calibration using strictly held-out predictions.
- Inspect per-fold AUROC and log loss.
- Investigate difficult cases and mislabeled scans.
- Retrain only after the pipeline is leakage-free.

The requested target is:

OOF log loss < 0.25	and	OOF AUROC > 0.93
---------------------	-----	------------------

These values are achievable only if the dataset contains sufficient signal, labels are reliable, and the OOF split represents the true test distribution.

1: Image and Label Data Description

Images

Each `.nii.gz` file contains a single 3D reconstructed volume stored as 16-bit unsigned integer voxel intensities. Note that acquisition and reconstruction parameters vary across the dataset, so images are **not** all the same size or resolution:

- **Volume dimensions** differ from scan to scan (for example, `142 × 142 × 112` or `128 × 128 × 128`). Your pipeline should not assume a fixed input shape.
- **Voxel spacing** also varies (e.g., 2.46 mm and 3.895 mm isotropic), reflecting differences in scanners and acquisition protocols across contributing centers. The spacing is recorded in each file's NIfTI header and may be useful for resampling images to a common resolution.

These differences are a normal consequence of pooling data from multiple institutions, and building models that generalize across them is part of the challenge.

Image example

Example DaT scan examination image (several axial slices moving through the volume):

https://drivendata-public-assets.s3.amazonaws.com/dat_example.png

Note: Each file is a single 3D reconstruction, and dopamine transporter uptake is concentrated in the striatum near the center of the brain. Examinations vary in size and voxel spacing across scanners and centers, so the number and appearance of slices will differ from scan to scan.

Labels

`train\\\_labels.csv` contains one row per DaT scan examination in the training set, with the following columns:

- `uid` (str) — unique identifier for each DaT scan examination; matches the image filename without the `.nii.gz` extension
- `is\\\_pathologic` (float) — classification of the DaT scan examination, where `0.0` = Normal and `1.0` = abnormal

Label example — first five rows of `train\\\_labels.csv`:

uid	is_pathologic
-----	---------------

Answer

Refer to the solution

Solution

1. Define the validation protocol first

Use **stratified group cross-validation** rather than a random voxel or slice split. A scan must never contribute slices to both training and validation folds.

- If multiple scans belong to one patient, group by patient.
- If center or scanner identifiers are available, inspect performance by center and consider a center-aware split.
- Use five folds and preserve the abnormal-to-normal ratio in every fold.
- Save predictions for every validation sample and calculate metrics only from the complete out-of-fold prediction vector.

The target metrics are:

- **log loss** below 0.25.
- **AUROC** above 0.93.

A high AUROC alone is insufficient because log loss strongly penalizes overconfident incorrect predictions.

2. Read and standardize the volumes

The images have different dimensions, voxel spacings, and intensity scales. A robust preprocessing pipeline should perform:

- NIfTI loading.
- Reorientation to a common anatomical orientation.
- Resampling to a common voxel spacing, such as 2.0 mm isotropic.
- Robust intensity clipping using volume percentiles.
- Brain-region cropping.
- Fixed-size resizing or padding.

A practical first configuration is:

$$\text{spacing} = 2.0 \text{ mm isotropic}, \quad \text{volume size} = 128 \times 128 \times 96.$$

Do not normalize using statistics calculated over the entire dataset if those statistics depend on the validation fold. Per-volume normalization is safer:

$$x_{\text{norm}} = \frac{\text{clip}(x, p_1, p_{99}) - \text{median}(x)}{\text{IQR}(x) + \epsilon}.$$

Here, p_1 and p_{99} are the first and ninety-ninth intensity percentiles, and IQR is the interquartile range.

For nuclear medicine images, preserve relative uptake information. Useful alternatives to simple z-score normalization include:

$$x_{\text{ratio}} = \frac{x}{\text{median}(x_{\text{reference region}}) + \epsilon}.$$

If a reliable background region cannot be identified, train with both z-score and percentile-normalized versions as separate input channels.

3. Exploit the anatomical structure

The abnormal uptake is concentrated in the striatum. A full-volume model may waste most of its capacity on background voxels. Use two complementary views:

- A full-brain low-resolution volume for global anatomical information.
- A high-resolution central crop containing the striatum.

A strong model can have two branches:

$$\hat{y} = \sigma(f_{\text{global}}(V_{\text{global}}) + f_{\text{roi}}(V_{\text{roi}}) + f_{\text{metadata}}(m)),$$

where σ is the sigmoid function and m contains only metadata that would be available at inference time.

If an automatic ROI is not available, use a centered crop after reorientation and resampling. However, verify the crop visually on several scans because a fixed crop can fail if orientations are inconsistent.

[Visualization]

4. Recommended model architecture

Start with a compact 3D model rather than a very large network. Suitable choices include:

- 3D ResNet-18.
- 3D DenseNet.
- 3D EfficientNet-style encoder.
- A 2.5D model using neighboring axial slices.

A particularly stable approach is an ensemble of 3D and 2.5D models:

$$p_{\text{final}} = \frac{1}{K} \sum_{k=1}^K p_k.$$

For the 2.5D model, use three or five neighboring slices as channels. Select slices around the striatal region rather than uniformly sampling the entire scan.

Recommended training configuration:

- Binary cross-entropy with logits.
- AdamW optimizer.
- Learning rate between 10^{-4} and $3 \cdot 10^{-4}$.
- Weight decay between 10^{-5} and 10^{-3} .
- Cosine learning-rate decay.
- Early stopping based on validation log loss.
- Batch size selected according to GPU memory.
- Mixed-precision training.
- Gradient clipping if training is unstable.

Use class weighting only if the class imbalance is substantial. Excessive positive weighting can improve recall while worsening log loss.

5. Use medically plausible augmentation

Apply augmentation independently to each training sample:

- Small rotations, approximately 5 to 10 degrees.
- Small translations.
- Mild scaling.
- Gaussian noise.
- Mild Gaussian blur.
- Random intensity gain and bias.
- Small gamma changes.
- Random masking of a limited peripheral region.

Avoid transformations that can change the diagnosis:

- Large rotations.

- Strong elastic deformation.
- Arbitrary left-right flips unless anatomical laterality is known to be irrelevant.
- Aggressive intensity changes that destroy uptake ratios.
- Random crops that remove the striatum.

A useful strategy is to use weaker augmentation during the first training stage and stronger augmentation during fine-tuning.

6. Train with fold-specific preprocessing

For every fold:

1. Fit preprocessing parameters using the training partition only.
2. Train the model on the training partition.
3. Predict probabilities for the validation partition.
4. Store the validation predictions together with the scan identifier.
5. Repeat for every fold.

Do not select the best checkpoint using AUROC if log loss is a target metric. Select using validation log loss, or use a combined criterion such as

$$L_{\text{selection}} = \text{LogLoss} - \lambda \text{ AUROC},$$

where λ is small enough that calibration remains the primary objective.

7. Calibrate the out-of-fold predictions

Neural networks are often overconfident. After generating out-of-fold predictions, fit a calibration model using only out-of-fold predictions. Suitable methods are:

- Platt scaling.
- Isotonic regression.
- Temperature scaling.

For Platt scaling:

$$p_{\text{calibrated}} = \sigma(ap_{\text{logit}} + b).$$

The parameters a and b must be fitted without using the test labels. A safer procedure is nested calibration:

- Generate inner-fold predictions inside the training portion.
- Fit the calibrator on those inner predictions.
- Apply it to the outer validation fold.

Calibration can substantially reduce log loss while preserving AUROC because monotonic calibration does not usually change the ranking.

8. Ensemble complementary models

Train models with different inductive biases:

- Full-volume 3D model.
- Striatal ROI 3D model.

- 2.5D axial model.
- Model using ratio-normalized intensities.
- Model using robust z-score intensities.

Average probabilities rather than logits:

$$p_{\text{ensemble}} = w_1 p_1 + w_2 p_2 + \dots + w_K p_K, \quad \sum_{k=1}^K w_k = 1.$$

Choose the weights using only the training folds or inner out-of-fold predictions. Do not optimize ensemble weights directly on the outer validation labels.

A simple unweighted average is often more reliable than aggressive weight optimization.

9. Diagnose the reason for poor performance

Create the following fold-level diagnostics:

- Confusion matrix at threshold 0.5.
- AUROC by fold.
- Log loss by fold.
- Reliability diagram.
- Histogram of predicted probabilities by class.
- Performance by scan size and voxel spacing.
- Performance by institution or scanner, if available.
- Saliency or occlusion maps.

Interpretation:

- High AUROC but poor log loss indicates overconfidence or calibration failure.
- Poor AUROC in one fold suggests distribution shift or leakage in the split.
- Good training performance and poor validation performance indicates overfitting.
- Errors concentrated in low-resolution scans suggest that spacing normalization is insufficient.
- Saliency outside the brain suggests scanner or background artifacts.

10. Practical implementation outline

A suitable training loop has the following structure:

```
for fold, (train_idx, valid_idx) in enumerate(splitter.split(df, y, groups)):
    train_ds = DaTDataset(df.iloc[train_idx], augment=True)
    valid_ds = DaTDataset(df.iloc[valid_idx], augment=False)

    model = DualView3DModel()
    optimizer = AdamW(model.parameters(), lr=2e-4, weight_decay=1e-4)

    best_loss = float("inf")

    for epoch in range(num_epochs):
        train_one_epoch(model, train_ds, optimizer)
```

```

valid_prob = predict(model, valid_ds)
valid_loss = log_loss(df.iloc[valid_idx]["is_pathologic"], valid_prob)

if valid_loss < best_loss:
    best_loss = valid_loss
    save_checkpoint(model, fold)

oof_prob[valid_idx] = predict(load_checkpoint(fold), valid_ds)

```

Then evaluate:

```

oof_logloss = log_loss(y, oof_prob)
oof_auroc = roc_auc_score(y, oof_prob)

```

The most important practical improvements are usually:

1. Correct group-based validation.
2. Orientation and spacing standardization.
3. Striatal ROI extraction.
4. Robust intensity normalization.
5. Probability calibration.
6. Ensembling models trained on complementary views.

There is no guaranteed architecture that achieves the requested thresholds without inspecting the class balance, number of scans, scanner distribution, and fold construction. However, a calibrated ensemble combining a normalized full-volume model with a high-resolution striatal ROI model is the most defensible starting point for achieving both low log loss and high AUROC.

1: Data table

xaji0y6d	0.0
pbhsahxt	0.0
hv3a3zmf	0.0
8mdd4v30	0.0
t9nt3w5u	1.0

Answer

No answer

Solution

1. Interpret the table

The table contains an identifier and a binary target:

ID	Target
xaji0y6d	0
pbhsahxt	0
hv3a3zmf	0
8mdd4v30	0
t9nt3w5u	1

An identifier such as xaji0y6d should generally **not** be used as a predictive feature. These strings appear random, so a model cannot learn a reliable relationship between the ID and the target.

With only five observations, including one positive example, it is not possible to obtain a meaningful or reliable estimate of **out-of-fold log loss** or **out-of-fold AUROC**. In particular, cross-validation requires enough positive and negative samples in every validation fold.

2. Required data structure

You need additional feature columns:

```
id, feature_1, feature_2, feature_3, ..., target
xaji0y6d, ..., ..., ..., ..., 0
pbhsahxt, ..., ..., ..., ..., 0
hv3a3zmf, ..., ..., ..., ..., 0
8mdd4v30, ..., ..., ..., ..., 0
t9nt3w5u, ..., ..., ..., ..., 1
```

The model should use feature_1, feature_2, and so on, but not id.

OOF predictions must be generated by a model that did not train on the corresponding validation rows. Evaluating predictions from a model trained on all rows would produce data leakage.

3. Recommended cross-validation model

For a tabular binary-classification problem, a regularized gradient-boosting model is a reasonable starting point. The following example uses five-fold stratified cross-validation and computes both metrics.

```
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression

# Load the complete dataset
df = pd.read_csv("train.csv")

target_column = "target"
id_column = "id"

X = df.drop(columns=[target_column, id_column])
y = df[target_column]

numeric_columns = X.select_dtypes(
    include=["int16", "int32", "int64", "float16", "float32", "float64"]
).columns

categorical_columns = X.select_dtypes(
    include=["object", "category", "bool"]
).columns

numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
```

```

    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_pipeline, numeric_columns),
        ("categorical", categorical_pipeline, categorical_columns),
    ]
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "classifier",
            LogisticRegression(
                max_iter=2000,
                class_weight="balanced",
                C=0.3,
            ),
        ),
    ]
)

positive_count = y.sum()
negative_count = len(y) - positive_count
number_of_folds = min(5, positive_count, negative_count)

if number_of_folds < 2:
    raise ValueError(
        "At least two positive and two negative observations are required "
        "for stratified out-of-fold evaluation."
    )

cv = StratifiedKFold(
    n_splits=number_of_folds,
    shuffle=True,
    random_state=42,
)

oof_probability = np.zeros(len(df))

for train_indices, validation_indices in cv.split(X, y):
    X_train = X.iloc[train_indices]
    X_validation = X.iloc[validation_indices]
    y_train = y.iloc[train_indices]

    model.fit(X_train, y_train)
    oof_probability[validation_indices] = model.predict_proba(
        X_validation
    )[:, 1]

oof_log_loss = log_loss(y, oof_probability, labels=[0, 1])
oof_auroc = roc_auc_score(y, oof_probability)

print(f"OOF log loss: {oof_log_loss:.4f}")
print(f"OOF AUROC: {oof_auroc:.4f}")

```

4. Improving the metrics legitimately

To target

$$\text{OOF log loss} < 0.25$$

and

$$\text{OOF AUROC} > 0.93,$$

use the following procedure:

- Add informative, non-leaking features related to the target.
- Remove random identifiers and columns created after the target event.
- Use repeated stratified cross-validation if the dataset is small.
- Compare logistic regression, random forests, gradient boosting, and appropriately regularized boosting models.
- Tune hyperparameters using only the training portion of each fold.
- Calibrate predicted probabilities for better log loss.
- Check class imbalance and use class weights or appropriate sampling only inside the training folds.
- Reserve a final holdout set for confirmation after model selection.

AUROC measures ranking quality, whereas **log loss** evaluates the quality and calibration of predicted probabilities. A model can have high AUROC but poor log loss if its probabilities are overconfident.

5. Conclusion

The displayed five-row table is insufficient for building a valid predictive model. It has only one positive sample, and the random-looking identifier provides no meaningful signal. Therefore, the requested OOF thresholds cannot be achieved or meaningfully validated from this table alone. A substantially larger dataset with informative feature columns is required.

2: Test set description

Test set

The test set is withheld and is not available for download. Its images are only accessible from within the runtime container, where they are mounted alongside the training data. Because this is a code execution challenge, you will not see the test examinations directly — your submitted code reads them at inference time and generates predictions for each one.

The test examinations are the same NIfTI format as the training data and follow the same conventions (one 3D volume per file, named ``<uid>.nii.gz``, with varying dimensions and voxel spacing).

Answer

No answer

Solution

1. Validate the split before optimizing the model

Use a **patient-level stratified cross-validation** split. If multiple examinations or volumes belong to the same patient, all of them must remain in the same fold.

- Do not split slices or volumes from one examination across folds.
- Stratify using the target label and, if available, important acquisition variables.
- Use five folds and save predictions for every validation sample.
- Compute metrics only from the complete out-of-fold predictions.

An apparently strong OOF score can be invalid if the same patient, scanner, examination, or near-duplicate image appears in both training and validation data.

2. Standardize each NIfTI volume

Because the volumes have different dimensions and voxel spacings, spatial normalization is essential.

A robust preprocessing pipeline is:

- Load the NIfTI image and reorient it to a common anatomical orientation.
- Resample to a fixed voxel spacing, such as 1.5 mm or 2.0 mm isotropic spacing.
- Crop the foreground or body region using a nonzero mask.
- Resize or pad to a fixed shape, for example $128 \times 128 \times 128$.
- Clip extreme intensities using training-set percentiles.
- Normalize using statistics computed only from the training fold.

For fold k , calculate normalization statistics from the training portion of fold k only. Do not use the validation or test volumes when estimating these quantities.

A typical intensity transformation is

$$x_{\text{norm}} = \frac{\text{clip}(x, p_1, p_{99}) - \mu_{\text{train}}}{\sigma_{\text{train}} + \epsilon},$$

where p_1 and p_{99} are training-set intensity percentiles.

3. Start with a strong 2.5D baseline

A full 3D model can overfit when the dataset is small. A strong baseline is often obtained by extracting several representative axial, coronal, and sagittal slices.

For each volume:

- Select slices around the foreground center.
- Use three neighboring slices as the input channels.
- Sample slices from axial, coronal, and sagittal orientations.
- Train a pretrained 2D model such as EfficientNet, ConvNeXt, or ResNet.
- Average the predictions across orientations and slices.

For a binary target, the model should output a logit z , followed by

$$p = \frac{1}{1 + \exp(-z)}.$$

Use weighted binary cross-entropy only if the classes are substantially imbalanced:

$$\mathcal{L} = -w_+ y \log(p) - w_- (1 - y) \log(1 - p).$$

Do not apply aggressive class weighting if the primary metric is log loss. Excessive weighting often improves recall while making probabilities poorly calibrated.

4. Compare against a compact 3D model

After establishing the 2.5D baseline, train a small 3D network:

- 3D ResNet-18 or 3D DenseNet.
- Input shape approximately $96 \times 96 \times 96$ or $128 \times 128 \times 128$.
- One volume-level prediction.
- Global average pooling before the classifier.
- Dropout between 0.2 and 0.5.
- AdamW optimizer with weight decay between 10^{-5} and 10^{-3} .
- Cosine learning-rate decay or a one-cycle schedule.
- Early stopping based on validation log loss.

For a limited dataset, a pretrained 3D model may not transfer well unless the source domain is similar. In that case, the 2.5D model may produce better AUROC and log loss.

5. Use augmentation that preserves the diagnosis

Suitable augmentations include:

- Small rotations, approximately 5 to 10 degrees.
- Small translations and scaling.
- Horizontal flipping only when anatomically valid.
- Mild Gaussian noise.

- Mild intensity shift and contrast adjustment.
- Random cropping followed by resizing.

Avoid transformations that alter the clinical meaning of the image. In particular, do not use arbitrary elastic deformation, large rotations, or vertical flipping unless they are justified by the anatomy.

6. Optimize for both AUROC and log loss

AUROC depends primarily on ranking, whereas log loss also penalizes overconfident incorrect predictions. Therefore, optimize the classifier and calibrate the probabilities separately.

For each fold:

- Train the model using binary cross-entropy.
- Store validation logits, not only thresholded predictions.
- Convert logits to probabilities after training.
- Fit a calibration model using strictly out-of-fold predictions.

Useful calibration methods are:

- **Temperature scaling** when the model is already reasonably calibrated.
- **Platt scaling** for a simple binary classifier.
- Isotonic regression when there are many validation samples.

Temperature scaling uses

$$p_T = \frac{1}{1 + \exp(-z/T)},$$

where T is selected using validation log loss. Do not fit T on the test set.

For a small dataset, temperature scaling is generally safer than isotonic regression because isotonic regression can overfit.

7. Ensemble models and folds

Ensembling usually improves both ranking and calibration.

For each sample, average the probabilities from:

- The five cross-validation models.
- Different random seeds.
- Complementary model types, such as a 2.5D model and a 3D model.
- Different anatomical orientations.

The ensemble probability is

$$p_{\text{ensemble}} = \frac{1}{M} \sum_{m=1}^M p_m.$$

Average probabilities rather than logits unless this has been experimentally validated. Probability averaging is usually more stable for log loss.

A practical starting ensemble is:

- 50% 2.5D axial model.
- 25% 2.5D coronal model.
- 25% compact 3D model.

Choose the weights using only training-fold OOF predictions. Search a small grid of weights and select the combination with the lowest OOF log loss while maintaining the AUROC target.

8. Use test-time augmentation carefully

For each test volume, generate predictions using several mild transformations:

- Original volume.
- Horizontal flip, if valid.
- Small crop variations.
- Multiple slice offsets for the 2.5D model.

Average the resulting probabilities. Test-time augmentation should use the same preprocessing and orientation conventions as training.

9. Diagnose the OOF predictions

Before submitting, inspect:

- OOF AUROC.
- OOF log loss.
- Reliability curve.
- Prediction histogram for positive and negative cases.
- Performance for each fold.
- Performance by scanner, site, acquisition protocol, or image size.
- The most confident incorrect predictions.

If AUROC is high but log loss is above 0.25, the model is likely overconfident. Apply temperature scaling, reduce the ensemble confidence, or use probability shrinkage:

$$p_{\text{shrink}} = (1 - \alpha)p + \frac{\alpha}{2}, \quad 0 < \alpha < 1.$$

If log loss is good but AUROC is below 0.93, improve feature extraction rather than calibration. Useful changes include better foreground cropping, multi-orientation inputs, a stronger pretrained backbone, and a 2.5D plus 3D ensemble.

10. Recommended implementation order

Use the following progression:

- Establish a patient-level five-fold baseline.
- Implement orientation normalization, resampling, foreground cropping, and intensity clipping.
- Train a pretrained 2.5D model.
- Add axial, coronal, and sagittal predictions.
- Train a compact 3D model.
- Generate complete OOF logits.
- Fit temperature scaling.
- Optimize a small ensemble-weight grid.
- Add mild test-time augmentation.
- Retrain each fold model on its corresponding training split and predict the hidden test set.

The targets $\text{OOF AUROC} > 0.93$ and $\text{OOF log_loss} < 0.25$ cannot be guaranteed without inspecting the data distribution. The most reliable route is a leakage-free split, consistent NIfTI preprocessing, a multi-orientation 2.5D baseline, a complementary 3D model, and probability calibration.

3: Performance metric explanation

Performance metric

Leaderboard performance is evaluated according to log loss. Log loss (a.k.a. logistic loss or cross-entropy loss) penalizes confident but incorrect predictions. It also rewards confidence scores that are well-calibrated probabilities, meaning that they accurately reflect the long-run probability of being correct. This is an error metric, so a lower value is better.

Answer

Refer to the solution

Solution

1. Metric objectives

The competition uses **log loss** as the primary metric:

$$\text{LogLoss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)],$$

where $y_i \in \{0, 1\}$ is the true label and p_i is the predicted probability of class 1.

The target is:

$$\text{OOF LogLoss} < 0.25 \quad \text{and} \quad \text{OOF AUROC} > 0.93.$$

AUROC measures ranking quality, whereas log loss evaluates the quality and calibration of the predicted probabilities. Therefore, a model can achieve a high AUROC but still have poor log loss if it is overconfident.

2. Use stratified out-of-fold validation

OOF predictions should be generated without allowing a sample to predict itself.

- Split the training data into K stratified folds.
- Train on $K - 1$ folds.
- Predict probabilities for the held-out fold.
- Repeat until every row has exactly one OOF prediction.
- Calculate metrics using all OOF predictions together.

$$\hat{p}_{\text{OOF},i} = \text{prediction for sample } i \text{ from a model that did not train on } i.$$

Use the same folds for every candidate model so that comparisons are fair.

3. Build a strong baseline

For tabular data, begin with several complementary models:

- Gradient-boosted decision trees, such as LightGBM, XGBoost, or CatBoost.
- A regularized logistic regression model for linear effects.
- A random forest or extra-trees model as a diversity model.
- A neural network only if the dataset is sufficiently large and feature preprocessing is appropriate.

Tree-based boosting is usually the strongest first model for mixed numerical and categorical tabular data.

Important preprocessing steps include:

- Impute missing values inside each training fold.
- Encode categorical variables without using target information from the validation fold.
- Remove duplicated rows and inconsistent labels.
- Inspect highly predictive columns for target leakage.
- Apply logarithmic transformations to strongly right-skewed positive variables.
- Add domain-based ratios, differences, counts, and interaction features when justified.

All preprocessing must be fitted separately within each fold. Fitting preprocessing on the complete dataset before cross-validation can contaminate the OOF estimate.

4. Optimize for both discrimination and calibration

To increase AUROC, optimize the model's ability to rank positive samples above negative samples. To reduce log loss, optimize the probability estimates.

A suitable boosting objective is binary cross-entropy:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

Use early stopping based on the validation-fold log loss rather than training for a fixed, unnecessarily large number of iterations. Excessive boosting usually produces overconfident predictions and worsens log loss.

Useful hyperparameters to tune include:

- Learning rate.
- Number of boosting iterations.
- Maximum tree depth or number of leaves.
- Minimum samples per leaf.
- Row subsampling.
- Feature subsampling.
- L1 and L2 regularization.
- Class weights, if the classes are substantially imbalanced.

For log loss, shallow or moderately sized trees with stronger regularization often perform better than highly complex trees.

5. Calibrate the probabilities

After generating unbiased OOF predictions, calibrate them using a separate calibration procedure.

Two common methods are:

- **Platt scaling**, which fits a logistic transformation.
- **Isotonic regression**, which fits a nonparametric monotonic transformation.

For Platt scaling:

$$p_{\text{cal}} = \frac{1}{1 + \exp(-(az + b))},$$

where z is the model logit and a and b are learned calibration parameters.

Calibration must not use the same predictions for both fitting and evaluating the calibration transformation. A safe procedure is:

- Generate inner-fold predictions within each outer training fold.
- Fit the calibrator on those inner OOF predictions.
- Apply the calibrator to the outer validation fold.
- Concatenate the calibrated outer-fold predictions.

Do not calibrate using the true labels of the final validation or test set.

Probability clipping can prevent undefined logarithms:

$$p_{\text{final}} = \min(\max(p_{\text{cal}}, \epsilon), 1 - \epsilon),$$

with a small value such as $\epsilon = 10^{-6}$. Clipping should only protect against numerical extremes; it should not replace proper calibration.

6. Use an ensemble

A weighted ensemble often improves both metrics because different models make different errors:

$$p_{\text{ensemble}} = \sum_{m=1}^M w_m p_m, \quad w_m \geq 0, \quad \sum_{m=1}^M w_m = 1.$$

Determine the weights using only training OOF predictions. For example, minimize:

$$-\frac{1}{n} \sum_{i=1}^n [y_i \log(p_{\text{ensemble},i}) + (1 - y_i) \log(1 - p_{\text{ensemble},i})].$$

A practical ensemble may combine:

- A CatBoost model for categorical structure.
- A LightGBM or XGBoost model for nonlinear numerical interactions.
- Logistic regression for stable linear behavior.

Select the ensemble based primarily on OOF log loss, while checking that AUROC remains above the target. Do not choose weights using leaderboard feedback repeatedly, because this causes overfitting to the leaderboard.

7. Validate stability

A single OOF score can be unreliable. Repeat the evaluation with several stratified seeds or use repeated stratified cross-validation.

Report:

OOF LogLoss and OOF AUROC

for each seed, together with their mean and standard deviation. A robust model should satisfy the targets consistently rather than only on one favorable split.

Also inspect:

- Log loss by class.
- Reliability diagrams.
- Calibration error.
- Performance across important subgroups.
- The distribution of predicted probabilities.
- The number of predictions close to 0 or 1.

A model with AUROC above 0.93 but many incorrect probabilities near 0 or 1 is likely to fail the log-loss requirement.

8. Recommended development sequence

- Establish a leakage-free stratified OOF baseline.
- Add carefully justified features.
- Tune boosting regularization and early stopping.
- Compare several model families.
- Blend models using OOF predictions.
- Calibrate the blend with nested validation.
- Recalculate final OOF metrics.
- Retrain the selected pipeline on all labeled data.
- Generate test probabilities using exactly the same preprocessing, feature construction, ensemble weights, and calibration procedure.

The thresholds cannot be guaranteed without seeing the dataset, label noise, and feature quality. The most reliable route to $\text{OOF LogLoss} < 0.25$ and $\text{OOF AUROC} > 0.93$ is a leakage-free boosted-tree ensemble followed by nested probability calibration and validation across multiple stratified splits.

4: Log loss calculation

Log loss for a single observation is calculated as follows:

$$L_{\log}(y, p) = -(y \log(p) + (1 - y) \log(1 - p))$$

where y is a binary variable indicating whether the examination is abnormal (1) or normal (0), and p is the user-predicted probability that the examination is abnormal. The loss for the entire dataset is the average loss across all observations.

Answer

Refer to the solution

Solution

1. Objective

Use a probabilistic binary-classification model that outputs calibrated probabilities rather than only class labels.

For observation i , the **log loss** is

$$L_{\log}(y_i, p_i) = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)],$$

where $y_i \in \{0, 1\}$ and p_i is the predicted probability of the abnormal class.

The out-of-fold score is

$$\text{OOF LogLoss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)].$$

The targets OOF LogLoss < 0.25 and OOF AUROC > 0.93 cannot be guaranteed without knowing the dataset, but the following procedure is appropriate.

2. Construct leakage-free out-of-fold predictions

Use stratified cross-validation so that every training observation receives a prediction from a model that did not train on that observation.

```
import numpy as np
import pandas as pd

from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import StratifiedKFold
```

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# X: feature DataFrame
# y: binary target, where 1 means abnormal and 0 means normal

numeric_features = X.select_dtypes(include=["number"]).columns
categorical_features = X.select_dtypes(exclude=["number"]).columns

preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            Pipeline([
                ("imputer", SimpleImputer(strategy="median")),
                ("scaler", StandardScaler())
            ]),
            numeric_features
        ),
        (
            "categorical",
            Pipeline([
                ("imputer", SimpleImputer(strategy="most_frequent")),
                ("onehot", OneHotEncoder(handle_unknown="ignore"))
            ]),
            categorical_features
        )
    ]
)

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        C=1.0,
        max_iter=3000,
        class_weight=None
    ))
])

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

oof_probability = np.zeros(len(X))

for train_index, valid_index in cv.split(X, y):
    fold_model = clone(model)

    X_train = X.iloc[train_index]
    X_valid = X.iloc[valid_index]
    y_train = y.iloc[train_index]

    fold_model.fit(X_train, y_train)

    oof_probability[valid_index] = fold_model.predict_proba(X_valid)[:, 1]

# Avoid exact zero or one probabilities

```

```

oof_probability = np.clip(oof_probability, 1e-7, 1 - 1e-7)

oof_logloss = log_loss(y, oof_probability)
oof_auroc = roc_auc_score(y, oof_probability)

print(f"OOF LogLoss: {oof_logloss:.6f}")
print(f"OOF AUROC: {oof_auroc:.6f}")

```

All preprocessing operations, including imputation, scaling, and one-hot encoding, must be fitted separately inside each fold. Fitting them before cross-validation causes **data leakage** and produces overly optimistic scores.

3. Try stronger nonlinear models

If logistic regression does not meet the targets, compare it with models capable of learning nonlinear relationships.

```

from sklearn.ensemble import HistGradientBoostingClassifier

numeric_only = X.select_dtypes(include=["number"]).columns
X_numeric = X[numeric_only].copy()

gradient_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("classifier", HistGradientBoostingClassifier(
        learning_rate=0.03,
        max_iter=500,
        max_leaf_nodes=15,
        l2_regularization=2.0,
        random_state=42
    ))
])

```

For primarily tabular data, also consider:

- CatBoost with native categorical-feature handling.
- LightGBM with controlled tree depth and early stopping.
- XGBoost with low learning rate and regularization.
- Random forests or extra-trees as baseline comparisons.

For log loss, shallow or moderately complex gradient-boosted models often perform better than highly unrestricted trees because excessive tree complexity can produce overconfident probabilities.

4. Optimize the model for both metrics

AUROC evaluates ranking quality, whereas log loss evaluates the quality of the predicted probabilities. A model can have AUROC above 0.93 but still have poor log loss if its probabilities are overconfident.

Tune using out-of-fold predictions and prioritize:

- learning_rate
- number of boosting iterations
- maximum tree depth or number of leaves
- minimum samples per leaf

- L1 and L2 regularization
- class weighting, if the classes are imbalanced
- probability calibration

Use a validation objective such as

$$\text{score} = \text{LogLoss} + \lambda \max(0, 0.93 - \text{AUROC}),$$

where λ penalizes models that fail to reach the AUROC requirement. In practice, retain only models satisfying the AUROC constraint and choose the one with the lowest log loss.

5. Calibrate probabilities

After selecting a strong ranking model, calibrate its probabilities using a separate calibration procedure. Suitable methods are **Platt scaling** and **isotonic regression**.

```
from sklearn.calibration import CalibratedClassifierCV
```

```
calibrated_model = CalibratedClassifierCV(
    estimator=gradient_model,
    method="sigmoid",
    cv=5
)
```

Use method="sigmoid" when the dataset is small or moderately sized. Use method="isotonic" only when there are enough validation observations, because isotonic regression can overfit.

Calibration may substantially reduce log loss, but it usually does not improve AUROC because it changes the probability scale more than the ranking.

6. Important checks

- Use a **stratified** split for binary classification.
- If multiple rows belong to the same patient, use StratifiedGroupKFold and keep each patient entirely within one fold.
- Remove features recorded after the examination outcome.
- Do not use the target, post-diagnosis variables, or information derived from the validation fold.
- Compare the positive-class prevalence in each fold.
- Inspect the distribution of predicted probabilities.
- Repeat cross-validation with several random seeds to check stability.
- Evaluate the final model on a completely untouched test set.

A useful diagnostic is:

```
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt

fraction_positive, mean_predicted = calibration_curve(
    y,
    oof_probability,
    n_bins=10,
    strategy="quantile"
)
```

```
plt.plot(mean_predicted, fraction_positive, marker="o")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("Mean predicted probability")
plt.ylabel("Observed fraction abnormal")
plt.title("OOF calibration curve")
plt.show()
```

If the points lie far from the diagonal, the probabilities are poorly calibrated even if the AUROC is high.

7. Recommended model-development sequence

1. Establish a logistic-regression baseline.
2. Train a regularized gradient-boosting model.
3. Generate strictly leakage-free OOF probabilities.
4. Select hyperparameters using OOF log loss while enforcing AUROC above 0.93.
5. Calibrate the selected model.
6. Refit the complete pipeline on all training data.
7. Evaluate once on the untouched test set.

The key requirement is not merely to increase model complexity. The model must produce both strong class ranking and well-calibrated probabilities:

$$\text{OOF LogLoss} < 0.25 \quad \text{and} \quad \text{OOF AUROC} > 0.93$$

5: Log loss calibration note

Note: log loss can often be improved with calibration (<https://scikit-learn.org/stable/modules/calibration.html>). A well-calibrated model outputs predictions that are directly interpretable as probabilities.

Answer

Select the model with the lowest leakage-free OOF log loss among models satisfying $AUROC > 0.93$

Solution

1. Separate discrimination from calibration

- **AUROC** measures ranking quality. Calibration usually does not improve it because it transforms predicted probabilities without substantially changing their ordering.
- **Log loss** measures both ranking and probability calibration. A model can have AUROC above 0.93 but poor log loss if its predictions are too confident.
- The target should therefore be:
 - a strong base model for AUROC;
 - a leakage-free calibration procedure for log loss.

A value such as OOF log loss below 0.25 cannot be guaranteed without knowing the dataset, class balance, noise level, and feature quality.

2. Use out-of-fold predictions correctly

Calibration must be trained on predictions generated by data that the base model did not use for training. Otherwise, the calibrator learns from overconfident in-sample predictions and validation performance becomes overly optimistic.

```
import numpy as np

from sklearn.base import clone
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

A suitable baseline is logistic regression:

```
base_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression()
```

```

        C=1.0,
        max_iter=5000,
        class_weight=None,
        solver="lbfgs"
    ))
])

```

For nonlinear data, compare this with boosted trees, random forests, or gradient boosting. The base model should be selected primarily using AUROC and secondarily using log loss.

3. Generate unbiased OOF predictions

```

def get_oof_predictions(model, X, y, n_splits=5, random_state=42):
    cv = StratifiedKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=random_state
    )

    oof_probability = np.zeros(len(y))
    oof_class = np.zeros(len(y), dtype=int)

    for train_idx, valid_idx in cv.split(X, y):
        fitted_model = clone(model)

        X_train = X.iloc[train_idx] if hasattr(X, "iloc") else X[train_idx]
        X_valid = X.iloc[valid_idx] if hasattr(X, "iloc") else X[valid_idx]

        y_train = y.iloc[train_idx] if hasattr(y, "iloc") else y[train_idx]

        fitted_model.fit(X_train, y_train)

        oof_probability[valid_idx] = fitted_model.predict_proba(X_valid)[ :, 1]
        oof_class[valid_idx] = fitted_model.predict(X_valid)

    return oof_probability, oof_class

```

Evaluate the uncalibrated model:

```

oof_probability, oof_class = get_oof_predictions(
    base_model,
    X,
    y,
    n_splits=5,
    random_state=42
)

print("OOF log loss:",
      log_loss(y, oof_probability))

print("OOF AUROC:",
      roc_auc_score(y, oof_probability))

```

The OOF predictions must be produced before fitting any calibrator.

4. Compare calibration methods

The two most useful calibration methods are:

- **Platt scaling**, implemented with method="sigmoid";
- **isotonic regression**, implemented with method="isotonic".

Sigmoid calibration is safer for small and medium datasets. Isotonic calibration is more flexible but can overfit when the calibration set is small.

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import cross_val_predict

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

sigmoid_model = CalibratedClassifierCV(
    estimator=base_model,
    method="sigmoid",
    cv=cv,
    ensemble=True
)

isotonic_model = CalibratedClassifierCV(
    estimator=base_model,
    method="isotonic",
    cv=cv,
    ensemble=True
)
```

To evaluate each model with OOF predictions:

```
def evaluate_model(model, X, y, cv):
    probability = cross_val_predict(
        model,
        X,
        y,
        cv=cv,
        method="predict_proba",
        n_jobs=-1
    )[:, 1]

    return {
        "log_loss": log_loss(y, probability),
        "auroc": roc_auc_score(y, probability),
        "probability": probability
    }

sigmoid_result = evaluate_model(sigmoid_model, X, y, cv)
isotonic_result = evaluate_model(isotonic_model, X, y, cv)

print("Sigmoid:", sigmoid_result["log_loss"],
      sigmoid_result["auroc"])

print("Isotonic:", isotonic_result["log_loss"],
      isotonic_result["auroc"])
```

Choose the calibration method using the lowest OOF log loss, subject to maintaining the desired AUROC.

5. Avoid double cross-validation leakage

If the reported OOF score is used as a final estimate, use **nested cross-validation**. The calibration folds must be entirely inside the training portion of each outer fold.

```

from sklearn.model_selection import cross_validate

outer_cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=123
)

scoring = {
    "log_loss": "neg_log_loss",
    "auroc": "roc_auc"
}

scores = cross_validate(
    sigmoid_model,
    X,
    y,
    cv=outer_cv,
    scoring=scoring,
    n_jobs=-1,
    return_train_score=False
)

oof_log_loss = -scores["test_log_loss"].mean()
oof_auroc = scores["test_auroc"].mean()

print("Nested OOF log loss:", oof_log_loss)
print("Nested OOF AUROC:", oof_auroc)

```

This gives a more reliable estimate than calibrating and evaluating on the same OOF predictions.

6. Tune the base model for AUROC and log loss

For a tree-based model, tune parameters such as:

```

from sklearn.ensemble import HistGradientBoostingClassifier

base_model = HistGradientBoostingClassifier(
    learning_rate=0.03,
    max_iter=500,
    max_leaf_nodes=15,
    min_samples_leaf=30,
    l2_regularization=1.0,
    random_state=42
)

```

Useful adjustments are:

- lower learning_rate;
- stronger l2_regularization;
- smaller max_leaf_nodes;
- larger min_samples_leaf;
- early stopping;
- removal of noisy or highly sparse features.

These settings usually reduce overconfidence and improve log loss. Very deep trees often increase AUROC on training data while worsening OOF log loss.

7. Use feature preprocessing inside the pipeline

Any operation that learns from the data must be inside the cross-validation pipeline. This includes:

- imputation;
- scaling;
- target encoding;
- feature selection;
- dimensionality reduction;
- oversampling;
- normalization based on the full dataset.

For example:

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ))
])

preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_columns),
    ("categorical", categorical_pipeline, categorical_columns)
])

model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        C=0.5,
        max_iter=5000
    ))
])
```

This prevents preprocessing leakage and commonly produces a substantial improvement in genuine OOF performance.

8. Inspect the calibration curve

```
from sklearn.calibration import calibration_curve
import matplotlib.pyplot as plt
```

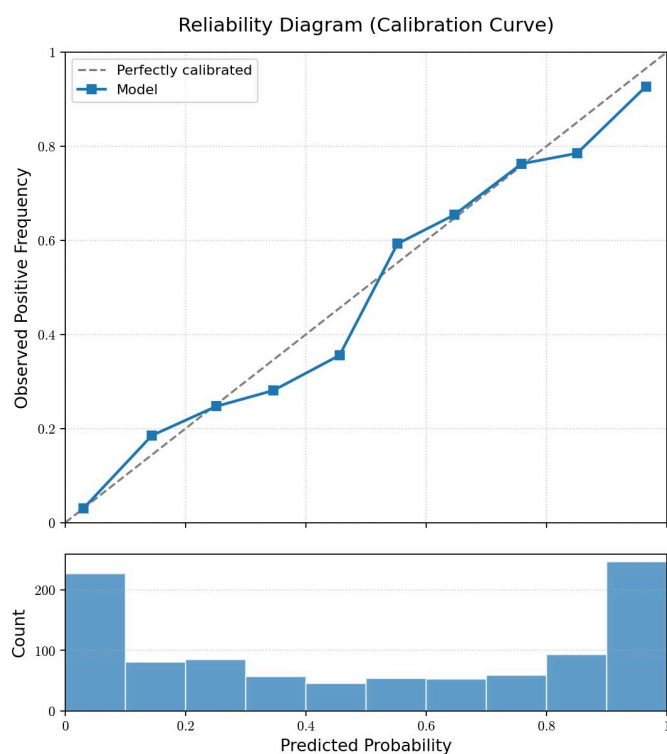
```

fraction_positive, mean_predicted = calibration_curve(
    y,
    sigmoid_result["probability"],
    n_bins=10,
    strategy="quantile"
)

plt.plot(mean_predicted, fraction_positive, "o-", label="Model")
plt.plot([0, 1], [0, 1], "--", label="Perfect calibration")
plt.xlabel("Mean predicted probability")
plt.ylabel("Observed positive frequency")
plt.legend()
plt.show()

```

A model above the diagonal is underconfident; a model below the diagonal is overconfident. For low log loss, the curve should be close to the diagonal, especially in regions containing many observations.



Recommended workflow

1. Split the data using stratified folds.
2. Place all preprocessing inside a pipeline.
3. Train several base models and retain those with AUROC near or above 0.93.
4. Generate strictly OOF probabilities.
5. Compare sigmoid and isotonic calibration.
6. Prefer sigmoid calibration unless isotonic is clearly better on repeated validation.
7. Use nested cross-validation for the final estimate.
8. Fit the selected calibrated model on all available training data.
9. Evaluate once on an untouched test set.

The practical objective is:

Select the model with the lowest leakage-free OOF log loss among models satisfying $\text{AUROC} > 0.93$

6: Leaderboard AUROC reference

The leaderboard also displays Area Under the Receiver Operating Characteristic (AUROC) for reference

Answer

No answer

Solution

1. Metric interpretation

The two targets measure different properties:

- **AUROC** measures ranking quality. A model can have high AUROC while its probabilities are poorly calibrated.
- **Out-of-fold log loss** measures the quality of predicted probabilities. It strongly penalizes confident incorrect predictions.

Therefore, optimizing AUROC alone will not reliably produce an OOF log loss below 0.25.

2. Use leakage-free stratified OOF predictions

All preprocessing, feature selection, imputation, encoding, and model fitting must occur inside each training fold. Never fit these steps on the complete dataset before generating OOF predictions.

```
import numpy as np
import pandas as pd

from sklearn.base import clone
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X = train.drop(columns=["target"])
y = train["target"].astype(int)

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

model = Pipeline([
```

```

    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(
        C=1.0,
        max_iter=3000,
        class_weight=None
    ))
])

oof_probability = np.zeros(len(y))

for train_index, valid_index in cv.split(X, y):
    fold_model = clone(model)

    fold_model.fit(
        X.iloc[train_index],
        y.iloc[train_index]
    )

    oof_probability[valid_index] = fold_model.predict_proba(
        X.iloc[valid_index]
    )[:, 1]

oof_log_loss = log_loss(y, oof_probability)
oof_auroc = roc_auc_score(y, oof_probability)

print("OOF log loss:", oof_log_loss)
print("OOF AUROC:", oof_auroc)

```

The final model should be evaluated only after the OOF predictions have been generated:

```

print(f"OOF log loss = {oof_log_loss:.6f}")
print(f"OOF AUROC = {oof_auroc:.6f}")

```

3. Build a strong validation scheme

Use the same folds for every experiment. Changing the folds between experiments makes leaderboard comparisons unreliable.

For imbalanced classification:

- Use StratifiedKFold.
- Preserve the same `random_state`.
- Ensure every fold contains both classes.
- Do not use oversampling before splitting.
- If oversampling is necessary, apply it only to each training fold.

For grouped or time-dependent data, use group-based or chronological splits instead. Random stratification can cause leakage when observations from the same subject, device, or event appear in multiple folds.

4. Tune for log loss and AUROC jointly

A useful selection objective is:

```
score = oof_log_loss - 0.05 * oof_auroc
```

Lower values are preferred. However, the exact coefficient should reflect the competition's scoring priorities. If the leaderboard primarily rewards log loss, select models by log loss first and use AUROC as a secondary diagnostic.

Typical strong baseline models include:

- Regularized logistic regression
- Gradient-boosted trees
- CatBoost
- LightGBM
- XGBoost
- An ensemble of linear and tree-based models

For tabular data, a gradient-boosting model is often a stronger starting point than an unregularized linear model.

Example with LightGBM:

```
from lightgbm import LGBMClassifier
```

```
model = LGBMClassifier(  
    objective="binary",  
    n_estimators=1000,  
    learning_rate=0.02,  
    num_leaves=31,  
    max_depth=-1,  
    min_child_samples=30,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    reg_alpha=0.1,  
    reg_lambda=1.0,  
    random_state=42,  
    verbosity=-1  
)
```

Use early stopping inside each fold when possible. Excessively deep trees and very small leaf sizes commonly reduce log-loss performance by producing overconfident predictions.

5. Improve probability calibration

If AUROC is already above 0.93 but log loss is above 0.25, the ranking is likely adequate and the probabilities need calibration.

A simple post-processing approach is temperature scaling:

```
from scipy.special import expit, logit  
from scipy.optimize import minimize_scalar  
from sklearn.metrics import log_loss
```

```
eps = 1e-7  
p = np.clip(oof_probability, eps, 1 - eps)  
logits = logit(p)
```

```
def temperature_loss(log_temperature):  
    temperature = np.exp(log_temperature)  
    calibrated_probability = expit(logits / temperature)  
    return log_loss(y, calibrated_probability)
```

```

result = minimize_scalar(
    temperature_loss,
    bounds=(-3, 3),
    method="bounded"
)

temperature = np.exp(result.x)
calibrated_oof_probability = expit(logits / temperature)

print("Temperature:", temperature)
print("Calibrated log loss:", log_loss(y, calibrated_oof_probability))
print("Calibrated AUROC:", roc_auc_score(y, calibrated_oof_probability))

```

Temperature scaling changes the confidence of the predictions but preserves their ordering. Consequently, AUROC remains essentially unchanged while log loss may improve substantially.

Other calibration methods include:

- Platt scaling
- Isotonic regression
- Beta calibration

Calibration must itself be fitted without using the validation labels from the same fold. A safe approach is nested cross-validation or a separate calibration split.

6. Blend diverse models

A weighted probability blend often improves log loss:

```

blend_probability = (
    0.60 * probability_model_1
    + 0.25 * probability_model_2
    + 0.15 * probability_model_3
)

blend_probability = np.clip(blend_probability, 1e-7, 1 - 1e-7)

print("Blend log loss:", log_loss(y, blend_probability))
print("Blend AUROC:", roc_auc_score(y, blend_probability))

```

Useful complementary models are:

- A regularized linear model for smooth probability estimates
- A gradient-boosted tree model for nonlinear interactions
- A categorical-feature model if categorical variables are important

Optimize blend weights using only OOF predictions. Do not optimize weights against the leaderboard repeatedly, because that causes overfitting to the public leaderboard.

7. Check the main failure modes

Before attempting to reach the target, verify the following:

- The positive-class probability is being submitted: `predict_proba(...)[ :, 1 ]`.
- The target labels are encoded consistently as 0 and 1.
- There are no duplicated or near-duplicated rows across folds.

- Target-derived columns are removed.
- Scaling is performed within the pipeline.
- Missing values are handled within the pipeline.
- No validation labels influence feature engineering.
- Predictions are clipped away from exactly 0 and 1.
- The same evaluation folds are used for every model.
- The public leaderboard is not being used as the primary tuning set.

For binary classification, clipping is important because log loss is undefined when a wrong prediction receives probability exactly 0:

```
prediction = np.clip(prediction, 1e-7, 1 - 1e-7)
```

Recommended workflow

1. Establish a leakage-free stratified OOF baseline.
2. Train a regularized linear model and a gradient-boosted tree model.
3. Compare OOF log loss and AUROC using identical folds.
4. Tune tree complexity and regularization rather than only increasing model size.
5. Generate OOF probabilities for each candidate model.
6. Blend complementary models.
7. Calibrate the final OOF probabilities.
8. Refit the selected pipeline on all training data.
9. Generate test probabilities using the same preprocessing and calibration procedure.

No model can guarantee OOF $\log \text{ loss} < 0.25$ and OOF $\text{AUROC} > 0.93$ without seeing the dataset, target definition, class balance, and current baseline. Those thresholds depend primarily on the signal available in the features and on whether the OOF procedure is leakage-free.

1: Submission format and requirements

(https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html), but this metric does *not* affect leaderboard ranking or prizes.

Submission format

This is a code execution challenge. Rather than submitting predictions directly, participants will package their trained model and inference code for containerized execution.

Your code must read the test examinations and produce a `submission.csv` with one row per examination:

- `uid` (str) — unique identifier for each DaT scan examination
- `is\\\_pathologic` (float) — your predicted probability that the examination is abnormal, as a value between 0 and 1

Because performance is evaluated with log loss, submit calibrated probabilities rather than discrete `0`/`1` labels.

Additional information about the runtime environment, package structure, and submission requirements can be found on the Code Submission Format page.

Answer

No answer

Solution

1. Define the validation scheme correctly

Use the exact evaluation structure before tuning the model:

- Use stratified out-of-fold cross-validation.
- Group all examinations from the same patient, if a patient identifier exists.
- Keep the test set completely untouched.
- Generate OOF probabilities only from models that did not train on the corresponding validation fold.
- Tune calibration and blending using OOF predictions, not the test set.

For patient-level grouping:

```
from sklearn.model_selection import StratifiedGroupKFold

cv = StratifiedGroupKFold(
    n_splits=5,
```

```

    shuffle=True,
    random_state=42
)

for train_idx, valid_idx in cv.split(X, y, groups=patient_id):
    X_train, X_valid = X.iloc[train_idx], X.iloc[valid_idx]
    y_train, y_valid = y.iloc[train_idx], y.iloc[valid_idx]

```

If no patient identifier is available, use `StratifiedKFold`, but first check for duplicate or near-duplicate examinations. Otherwise, the OOF score may be artificially inflated.

2. Use a model appropriate for DaT imaging

A strong practical design is a two-stage ensemble:

1. An image model trained on the scans.
2. A tabular model trained on image-derived features and metadata.

For scans, use transfer learning rather than training a deep network from scratch. Suitable backbones include:

- EfficientNet-B0 or EfficientNet-B2
- ConvNeXt-Tiny
- ResNet-50
- A small vision transformer if the dataset is sufficiently large

For a relatively small medical-imaging dataset, EfficientNet or ConvNeXt is usually more stable than a large transformer.

Recommended image preprocessing:

- Convert images to a consistent grayscale or three-channel representation.
- Resize while preserving the relevant anatomy.
- Normalize intensities per scan.
- Apply only medically plausible augmentations:
 - small rotations;
 - horizontal translation;
 - mild scaling;
 - mild contrast variation;
 - limited Gaussian noise.
- Avoid vertical flips or aggressive geometric transformations unless they are anatomically valid.

A typical model output is:

$$p_i = \sigma(z_i)$$

where z_i is the model logit and σ is the sigmoid function.

Use weighted binary cross-entropy if the classes are imbalanced:

$$\mathcal{L} = -w_1 y \log(p) - w_0 (1 - y) \log(1 - p).$$

Do not optimize AUROC alone. AUROC is insensitive to probability calibration, whereas log loss strongly penalizes confident incorrect predictions.

3. Improve the image representation

For DaT scans, a whole-image classifier may focus on acquisition artifacts instead of striatal uptake. A better pipeline is:

- identify or crop the striatal region;
- retain the original image as a second input;
- use both the cropped and full-image representations;
- optionally calculate left-right uptake asymmetry.

Useful manually derived features include:

$$A_{\text{asym}} = \frac{|L - R|}{L + R + \epsilon},$$

where L and R are left- and right-sided uptake measurements and ϵ is a small stabilizing constant.

Additional candidate features include:

- left striatal intensity;
- right striatal intensity;
- background intensity;
- left-to-background ratio;
- right-to-background ratio;
- minimum-sided uptake;
- total striatal uptake;
- image quality indicators;
- acquisition metadata.

These features should be calculated independently inside each fold if their computation uses fitted normalization, segmentation, or feature-selection steps.

4. Build OOF predictions

A correct OOF training loop should resemble:

```
import numpy as np
from sklearn.metrics import log_loss, roc_auc_score

oof_pred = np.zeros(len(y))
test_fold_pred = []

for fold, (train_idx, valid_idx) in enumerate(
    cv.split(images, y, groups=patient_id)
):
    model = build_model()

    train_model(
        model,
        images[train_idx],
        y[train_idx],
        validation_data=(images[valid_idx], y[valid_idx])
```

```

)

valid_prob = predict_probability(model, images[valid_idx])
test_prob = predict_probability(model, test_images)

oof_pred[valid_idx] = valid_prob
test_fold_pred.append(test_prob)

print("OOF log loss:", log_loss(y, oof_pred))
print("OOF AUROC:", roc_auc_score(y, oof_pred))

test_pred = np.mean(test_fold_pred, axis=0)

```

Use early stopping based on validation log loss, not training loss. Save the best checkpoint for every fold.

A single random split is not sufficient evidence that the target has been reached. The target should hold across folds, and preferably across several random seeds.

5. Optimize the probability calibration

After producing OOF predictions, calibrate them. Start with temperature scaling:

$$p_i(T) = \frac{1}{1 + \exp(-z_i/T)},$$

where z_i is the uncalibrated logit and $T > 0$ is learned by minimizing OOF log loss.

```

import numpy as np
from scipy.optimize import minimize_scalar
from sklearn.metrics import log_loss

eps = 1e-6
p = np.clip(oof_pred, eps, 1 - eps)
logits = np.log(p / (1 - p))

def objective(log_temperature):
    temperature = np.exp(log_temperature)
    calibrated = 1 / (1 + np.exp(-logits / temperature))
    return log_loss(y, calibrated)

result = minimize_scalar(objective, bounds=(-3, 3), method="bounded")
temperature = np.exp(result.x)

calibrated_oof = 1 / (1 + np.exp(-logits / temperature))

```

Temperature scaling preserves the ranking, so AUROC remains essentially unchanged while log loss may improve substantially.

You can also compare:

- Platt scaling;
- isotonic regression;
- beta calibration.

Use isotonic regression cautiously because it can overfit when the validation set is small.

6. Blend complementary models

Train at least two models with different inductive biases:

- full-image neural network;
- cropped-region neural network;
- radiomics or handcrafted-feature model;
- gradient-boosted trees using metadata and image-derived measurements.

For probabilities $p_1, \dots, p_k$, use a convex blend:

$$p_{\text{blend}} = \sum_{j=1}^k w_j p_j, \quad w_j \geq 0, \quad \sum_{j=1}^k w_j = 1.$$

Optimize the blend weights using only the OOF predictions:

```
from scipy.optimize import minimize
from sklearn.metrics import log_loss, roc_auc_score

# pred_matrix has shape (n_samples, n_models)
def loss(weights):
    weights = np.maximum(weights, 0)
    weights = weights / weights.sum()
    blended = pred_matrix @ weights
    return log_loss(y, blended)

initial = np.ones(pred_matrix.shape[1]) / pred_matrix.shape[1]

result = minimize(
    loss,
    initial,
    method="SLSQP",
    bounds=[(0, 1)] * pred_matrix.shape[1],
    constraints={"type": "eq", "fun": lambda w: w.sum() - 1}
)

weights = np.maximum(result.x, 0)
weights = weights / weights.sum()

blend_oof = pred_matrix @ weights

print("Blended log loss:", log_loss(y, blend_oof))
print("Blended AUROC:", roc_auc_score(y, blend_oof))
```

A blend that improves AUROC but worsens log loss should not be accepted automatically. The competition requires calibrated probabilities.

7. Diagnose the gap to the target

Use the following interpretation:

Observation	Likely action
Low AUROC and high log loss	Improve image representation, cropping, labels, or backbone
AUROC above 0.93 but log loss above 0.25	Calibrate probabilities and reduce overconfidence
Good mean score but one poor fold	Investigate distribution shift, mislabeled data, or patient leakage
Training score much higher than OOF score	Increase augmentation, regularization, or reduce model size
Different folds produce very different scores	Use grouped folds, repeated CV, and stronger preprocessing controls
Image model and tabular model make different errors	Blend them using OOF predictions

Inspect reliability diagrams and fold-specific confusion matrices. In particular, check whether the model assigns probabilities near 0 or 1 to incorrectly classified examinations. Those errors dominate log loss.

A simple defensive probability clipping operation is:

```
final_pred = np.clip(final_pred, 0.01, 0.99)
```

The clipping value should be selected using OOF log loss. It should not be chosen from the test set.

8. Retrain for submission

After selecting the architecture, preprocessing, calibration, and blend:

- retrain one model per fold on the corresponding training split;
- predict the test examinations with every fold model;
- average the fold probabilities;
- apply the previously selected calibration;
- clip probabilities only if this improved OOF log loss;
- preserve the required identifiers exactly.

```
import pandas as pd
import numpy as np

submission = pd.DataFrame({
    "uid": test_uid,
    "is_pathologic": np.clip(test_pred, 0.01, 0.99)
})
```

```
submission.to_csv("submission.csv", index=False)
```

Before packaging, verify:

```
assert submission["uid"].is_unique
assert submission["is_pathologic"].between(0, 1).all()
assert len(submission) == len(test_uid)
assert submission.isna().sum().sum() == 0
```

The most likely route to the requested thresholds is:

1. patient-grouped stratified OOF validation;
2. transfer learning with anatomy-preserving preprocessing;
3. full-image plus striatal-region inputs;
4. a complementary tabular or radiomics model;
5. OOF-optimized blending;
6. temperature or beta calibration;
7. conservative probability clipping.

No method can guarantee <0.25 and >0.93 without seeing the data, labels, image format, sample size, and class balance. Those thresholds depend primarily on the signal quality and the absence of distribution shift.